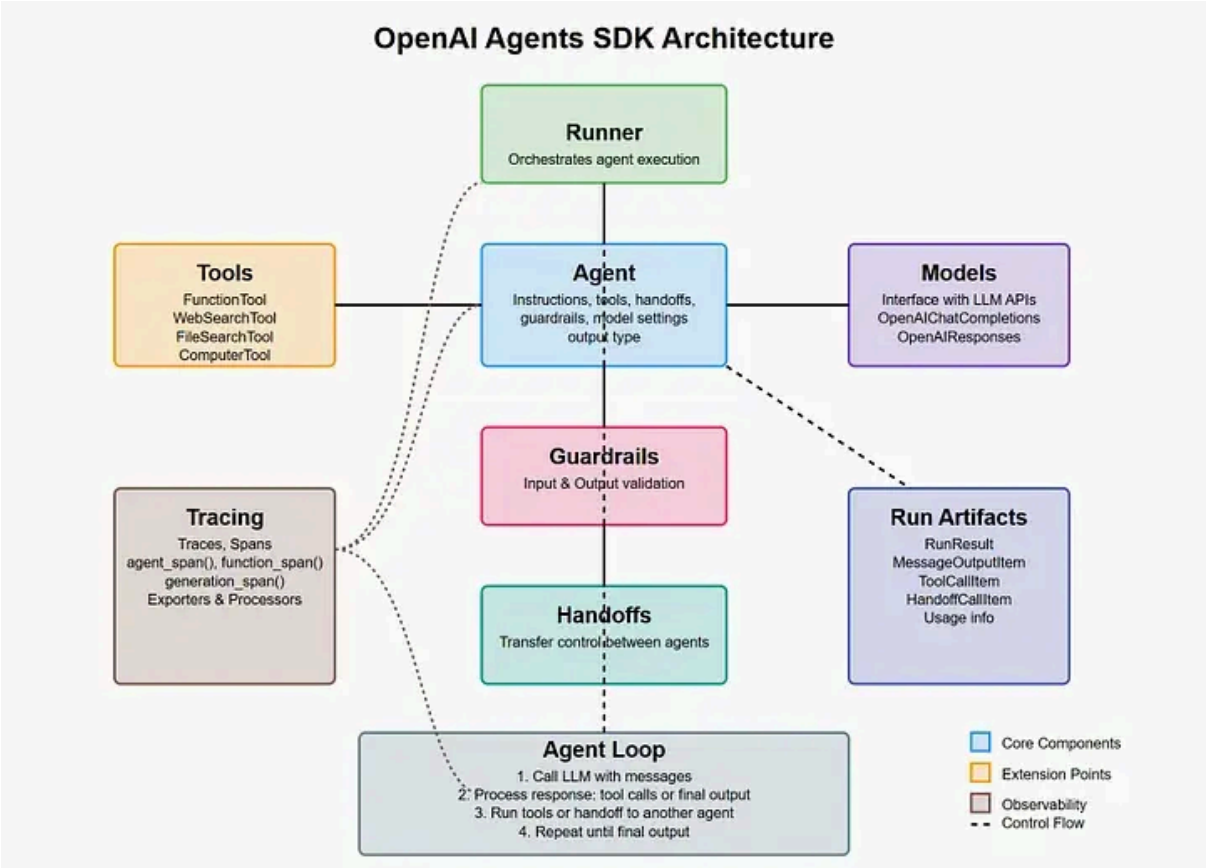
OpenAI Agents SDK Architecture

Detailed Professional Notes

This document provides a structured, technical explanation of the OpenAI Agents SDK architecture as shown in the diagram. It explains each component, its responsibility, and how the system operates as a whole in production environments.



---

1. Architectural Overview

The OpenAI Agents SDK is designed around separation of concerns. It distinguishes between:

- Cognitive reasoning (Model)
- Behavioral configuration (Agent)
- Execution orchestration (Runner)
- Capability extension (Tools)
- Safety and validation (Guardrails)
- Multi-agent coordination (Handoffs)
- Observability (Tracing)
- Execution outputs (Run Artifacts)
- Control flow (Agent Loop)

This modular structure allows the system to be scalable, debuggable, and production-ready.

---

2. Core Components

2.1 Agent

The Agent defines how the AI system should behave. It is a declarative configuration object that contains:

- Instructions (system-level behavior definition)

- Tool definitions available to the model
- Guardrails for validation
- Model configuration (e.g., temperature, model type)
- Output structure definition
- Handoff permissions

The Agent does not execute tasks directly. It defines the behavioral contract that the Runner enforces.

2.2 Runner

The Runner is the execution engine of the architecture. It is responsible for:

- Starting the agent execution
- Managing the iterative agent loop
- Calling the model
- Interpreting model outputs
- Executing tool calls
- Managing handoffs between agents
- Applying guardrails
- Producing final results

The Runner controls lifecycle, retries, termination conditions, and orchestration logic.

Without the Runner, the Agent configuration cannot operate.

2.3 Models

The Model represents the LLM interface. It is responsible for:

- Interpreting instructions
- Performing reasoning
- Selecting tools
- Generating structured outputs
- Producing handoff signals when necessary

The model does not execute tools or enforce safety. It only generates structured outputs that the Runner interprets and acts upon.

The separation ensures that reasoning and execution are distinct responsibilities.

---

3. Extension Points

3.1 Tools

Tools extend the capabilities of the agent beyond text generation. They allow interaction with external systems.

Examples include:

- Function tools (custom logic)
- Web search tools
- File search tools
- Computer or execution tools

The process is structured:

1. The model selects a tool and generates structured arguments.
2. The Runner validates the arguments.
3. The Runner executes the tool.
4. The result is returned to the model for further reasoning.

The model never executes tools directly. This ensures safety and control.

3.2 Handoffs

Handoffs allow one agent to transfer control to another agent.

This supports:

- Multi-agent systems
- Specialization of responsibilities
- Separation of reasoning domains

- Scalable architectures

For example:

- A planning agent may delegate execution to an executor agent.
- A research agent may hand off analysis to a data-processing agent.

The Runner manages handoffs and ensures state continuity.

---

4. Guardrails

Guardrails enforce validation and safety at runtime. They operate at multiple stages:

- Input validation
- Tool argument validation
- Intermediate output validation
- Final output validation

Guardrails help prevent:

- Invalid structured outputs
- Malformed tool arguments
- Policy violations

- Unsafe responses

They are critical in production systems where agents interact with real infrastructure.

---

5. Agent Loop

The Agent Loop is the core control mechanism of the system.

It performs iterative reasoning and execution until completion.

The loop includes:

1. Calling the model with current context.
2. Inspecting the response.
3. Executing tool calls if present.
4. Adding tool results back into context.
5. Handling handoffs if required.
6. Repeating until a final output is generated.

This loop transforms a stateless LLM into a stateful, goal-driven system.

---

6. Run Artifacts

Run Artifacts are structured outputs generated during execution. They include:

- Final result object
- Intermediate message outputs
- Tool call records
- Handoff records
- Usage statistics (token counts, cost information)

These artifacts enable:

- Auditability
- Replayability
- Cost tracking
- Debugging
- Monitoring

They provide a complete record of execution.

---

7. Tracing

Tracing is the observability layer of the architecture.

It captures:

- Agent execution spans

- Model generation spans
- Tool execution spans
- Handoff transitions
- Token usage
- Latency metrics

Tracing is essential for:

- Debugging complex workflows
- Monitoring production behavior
- Identifying bottlenecks
- Controlling costs
- Diagnosing failure cases

Without tracing, multi-step agents become difficult to operate at scale.

---

8. Control Flow

The architecture separates configuration relationships from runtime control flow.

Configuration defines:

- Which tools are available

- Which guardrails apply
- Which model is used

Runtime control flow determines:

- When tools are executed
- When the loop continues
- When agents hand off control
- When execution stops

This separation ensures flexibility and safety.

---

9. Responsibility Segmentation

The architecture assigns responsibilities clearly:

- Agent defines behavior.
- Model performs reasoning.
- Runner manages execution.
- Tools extend capability.
- Guardrails enforce safety.
- Handoffs enable collaboration.

- Tracing provides visibility.
- Run artifacts record outcomes.
- Agent loop ensures autonomy.

This segmentation reduces complexity and improves maintainability.

---

10. Production Considerations

In real-world deployments, additional policies must be defined:

- Maximum loop depth
- Timeout thresholds
- Tool execution limits
- Cost budgets
- Retry policies
- Access control boundaries

Proper observability and guardrail design are critical for enterprise deployment.

---

11. Architectural Strength

The OpenAI Agents SDK architecture addresses core limitations of standalone LLM systems:

- Adds controlled execution
- Enables multi-step reasoning
- Supports safe tool usage
- Provides validation mechanisms
- Enables multi-agent coordination
- Offers structured observability
- Supports cost monitoring

It converts a language model into a structured, autonomous execution framework suitable for production systems.

-